



Faculty of Electrical and Computer Engineering

Workshop Testing Cycle

Member:

Leonardo Zambrano Toala - ltzambra@espol.edu.ec

Teacher:

Dr. Carlos Mera Gómez

Guayaquil – Ecuador

2026



Part 1: Prepare the project

```

PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> python -m venv .venv
PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> .venv\Scripts\activate
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pip install -r requirements.txt -e .
Obtaining file:///C:/Users/leoza/Desktop/ESPOL/SeptimoSemestre/IngSW2/testing-cycle-roman-p1
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting pytest>=8.0
  Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
    386.5/386.5 kB 3.0 MB/s eta 0:00:00
Collecting pytest-cov>=5.0
  Downloading pytest_cov-7.1.0-py3-none-any.whl (22 kB)
Collecting colorama>=0.4
  Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Collecting iniconfig>=1.0.1
  Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Collecting packaging>=22
  Using cached packaging-26.2-py3-none-any.whl (100 kB)
Collecting pluggy<2,>=1.5
  Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting pygments>=2.7.2
  Using cached pygments-2.20.0-py3-none-any.whl (1.2 MB)
Collecting coverage[toml]>=7.10.6
  Downloading coverage-7.15.3-cp311-win_amd64.whl (224 kB)
    224.9/224.9 kB 6.9 MB/s eta 0:00:00
Building wheels for collected packages: roman
  Building editable for roman (pyproject.toml) ... done
  Created wheel for roman: filename=roman-0.1.0-0.editable-py3-none-any.whl size=1257 sha256=3b2d9663bb4bc879059c452ae50a854c8174ab1e626037bc1c9b960962d396ba
  Stored in directory: C:\Users\leoza\AppData\Local\Temp\pip-ephem-wheel-cache-fcvvuxn\wheels\61\57\16\e798f3977c041b34d1295a61271179d143a220bc2ea4074235
Successfully built roman
Installing collected packages: roman, pygments, pluggy, packaging, iniconfig, coverage, colorama, pytest, pytest-cov
Successfully installed colorama-0.4.6 coverage-7.15.3 iniconfig-2.3.0 packaging-26.2 pluggy-1.6.0 pygments-2.20.0 pytest-9.1.1 pytest-cov-7.1.0 roman-0.1.0

[notice] A new release of pip available: 22.3.1 -> 26.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pytest
===== test session starts =====
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 15 items

tests\test_converter.py ..... [100%]
  
```

Part 2: Audit the inherited suite

```

===== 15 passed in 0.07s =====
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pytest --cov=roman.converter --cov-branch --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 15 items

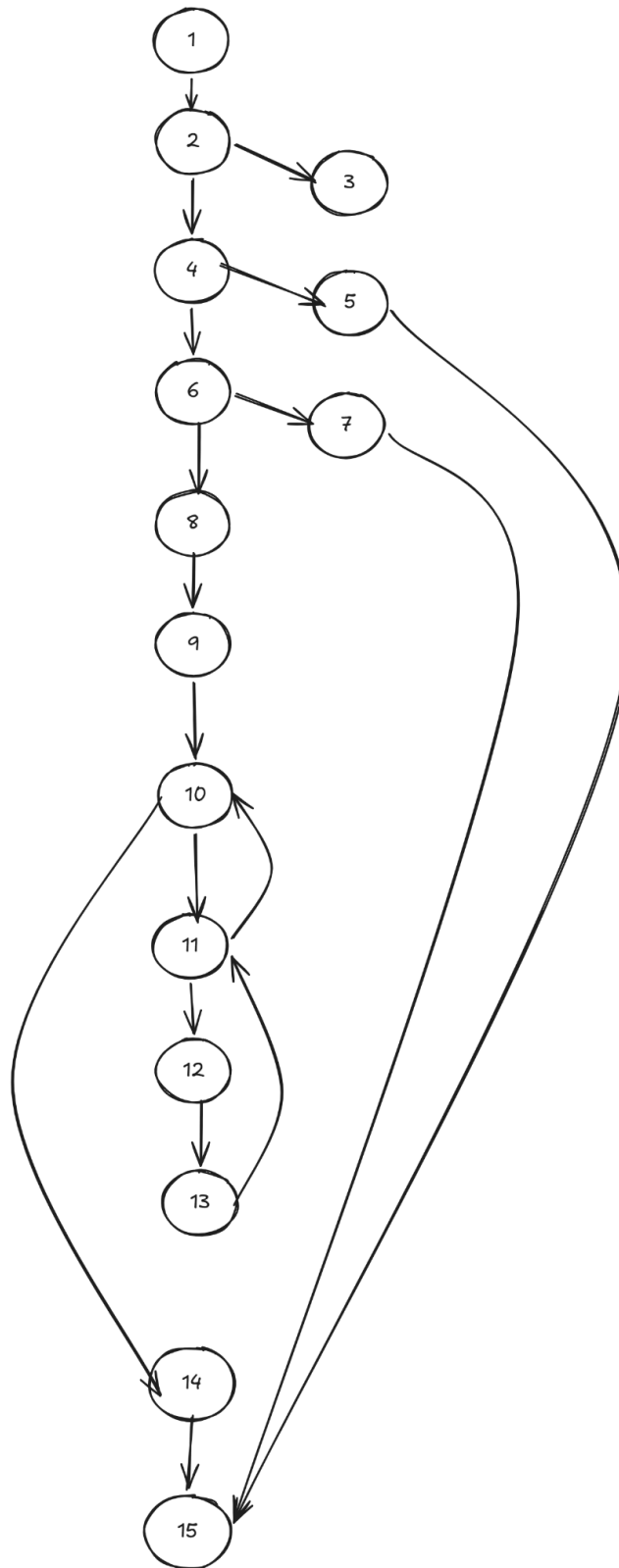
tests\test_converter.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.11.3-final-0

Name                Stmts   Miss Branch BrPart  Cover   Missing
-----
src\roman\converter.py  68      24      34      9    64%   42, 44, 46, 58, 61, 64, 72-74, 79, 83, 88, 92-96, 100-104, 108, 112
TOTAL                  68      24      34      9    64%

===== 15 passed in 0.12s =====
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
  
```

Part 3: Test at the unit level



Compute cyclomatic complexity $V(G) = E - N + 2$, and report the values of E and N you used. The result must match your graph.

- Number of Edges (E): 19
- Number of Nodes (N): 15

Using the formula for cyclomatic complexity:

$$V(G) = E - N + 2$$

$$V(G) = 19 - 15 + 2$$

$$V(G) = 6$$

List a basis set of $V(G)$ linearly independent paths, writing each one as a sequence of nodes.

- Path 1:
1 -> 2 -> 4 -> 6 -> 8 -> 9 -> 10 -> 14 -> 15
- Path 2:
1 -> 2 -> 3 -> 15
- Path 3:
1 -> 2 -> 4 -> 5 -> 15
- Path 4:
1 -> 2 -> 4 -> 6 -> 7 -> 15
- Path 5:
1 -> 2 -> 4 -> 6 -> 8 -> 9 -> 10 -> 11 -> 10 -> 14 -> 15
- Path 6:
1 -> 2 -> 4 -> 6 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 11 -> 10 -> 14 -> 15



Build the definition-use table for the variables of `to_roman`, marking each use as c-use or p-use. The variable remaining is redefined inside the loop, so include the pairs that this redefinition creates

	Category		
LINE	defintion	c-use	p-use
40	n		
41			n
42			
43			n
44			
45			n
46			
47	out		
48	remaining	n	
49	value, symbol		
50			remaining, value
51		out, symbol	
52	remaining	remaining, value	
53		out	

Identify du-pairs and their use (p- or c-)

Definition use pair	Variable(s)	
start line -> end line		
40->41		n
40->43		n
40->45		n



40->48	n	
47->51	out	
47->53	out	
48->50		remaining
48->52	remaining	
52->50		remaining
52->52	remaining	
49->50		value
49->52	value	
49->51	symbol	



Write unit tests until branch coverage of `src/roman/converter.py` reaches at least 85% coverage of what was captured in part 2.

```

72 def test_to_roman_rejects_non_integer():
73     with pytest.raises(RomanError, match="value must be an integer"):
74         to_roman("10")
75
76
77 def test_to_roman_rejects_boolean():
78     with pytest.raises(RomanError, match="value must be an integer"):
79         to_roman(True)
80
81
82 def test_to_roman_rejects_value_below_range():
83     with pytest.raises(RomanError, match="value must be >= 1"):
84         to_roman(0)
85
86
87 def test_to_roman_rejects_value_above_range():
88     with pytest.raises(RomanError, match="value must be <= 3999"):
89         to_roman(4000)
90
91
92 def test_to_roman_executes_multiple_loop_iterations():
93     assert to_roman(3888) == "MMMDCCLXXXVIII"
94
95
96 def test_from_roman_rejects_non_string():
97     with pytest.raises(RomanError, match="value must be a string"):
98         from_roman(10)
99
100
101 def test_from_roman_rejects_empty_string():
102     with pytest.raises(RomanError, match="empty string"):
103         from_roman("")
104
105
106 def test_from_roman_rejects_invalid_character():
107     with pytest.raises(RomanError, match="invalid roman character"):
108         from_roman("A")
109
110
111 def test_from_roman_accepts_valid_subtractive_pair():
112     assert from_roman("IX") == 9
  
```

```

(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> pytest --cov=roman.converter --cov-branch --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 29 items

tests\test_converter.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.11.3-final-0
Name                Stmts   Miss Branch BrPart  Cover   Missing
-----
src\roman\converter.py   68      6      34      0    90%   88, 92-96
TOTAL                   68      6      34      0    90%

===== 29 passed in 0.18s =====
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
  
```

Part 4: Test at the integration level

Unit tests exercise one function at a time. Integration tests combine two or more units and verify that they work together as a group.

Read section 7 of the specification. The functions `add_roman` and `subtract_roman` are built on top of `from_roman` and `to_roman`, and their result must be accepted by `is_valid_roman`.

```
(.venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> python -m pytest tests/test_integration.py -v
===== test session starts =====
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
plugins: cov-7.1.0
collected 2 items

tests/test_integration.py::test_add_roman_returns_canonical_and_valid_result FAILED [ 50%]
tests/test_integration.py::test_subtract_roman_returns_canonical_and_valid_result PASSED [100%]

===== FAILURES =====
_____ test_add_roman_returns_canonical_and_valid_result _____

    def test_add_roman_returns_canonical_and_valid_result():
        result = add_roman("II", "II")

>       assert result == "IV"
E       AssertionError: assert 'IIII' == 'IV'
E
E       - IV
E       + IIII

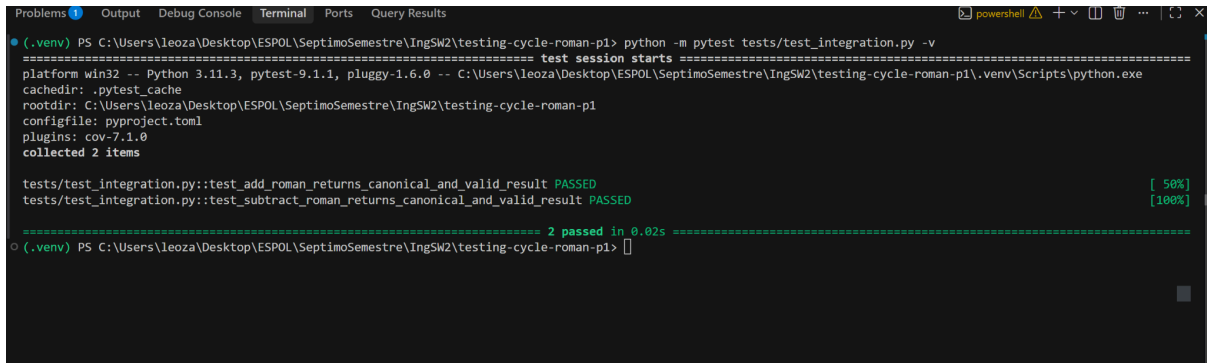
tests/test_integration.py:12: AssertionError

===== short test summary info =====
FAILED tests/test_integration.py::test_add_roman_returns_canonical_and_valid_result - AssertionError: assert 'IIII' == 'IV'
1 failed, 1 passed in 0.10s
```

The integration test verifies the collaboration between `add_roman`, `from_roman`, `to_roman`, and `is_valid_roman`. The test executes `add_roman("II", "II")`. Both operands are converted from Roman numerals to integers, added, and converted back to a Roman numeral. The produced result is then passed to `from_roman` and `is_valid_roman`. The expected canonical result according to section 7 of the specification is `'IV'`. However, the system returned `'IIII'`. The inherited unit tests did not detect this defect because they tested simple values such as I, II, III and V independently. They did not test the interaction of the arithmetic and conversion functions for the subtractive value four.

Correction and verification.

The defect was located in the `\_PAIRS` collection used by `to\_roman`. The Roman numeral `IV` was incorrectly associated with the integer value 5 instead of 4:



```
PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1> python -m pytest tests/test_integration.py -v
===== test session starts =====
platform win32 -- Python 3.11.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1
configfile: pyproject.toml
plugins: cov-7.1.0
collected 2 items

tests/test_integration.py::test_add_roman_returns_canonical_and_valid_result PASSED [ 50%]
tests/test_integration.py::test_subtract_roman_returns_canonical_and_valid_result PASSED [100%]

===== 2 passed in 0.02s =====
PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\testing-cycle-roman-p1>
```

After applying the correction, the integration test was executed again. The function `add_roman("II", "II")` returned the canonical result IV.

The result was successfully converted back to the integer 4 by `from_roman`, and it was accepted as valid by `is_valid_roman`.

All integration tests passed after the correction. The complete regression suite was also executed to verify that the change did not break the inherited tests.

Part 5: Test at the acceptance level

Write three acceptance criteria in Given / When / Then form, taken from ESPECIFICACION.md.

Acceptance Criterion 1 Given a valid canonical Roman numeral with leading and trailing whitespace When the user converts " IV " using `from\_roman` Then the system returns the integer 4.

Acceptance Criterion 2 Given the noncanonical Roman numeral "IIII" When the user attempts to convert it using `from\_roman` Then the system raises `RomanError` because the canonical representation of 4 is "IV".

Acceptance Criterion 3 Given a value that is not a string When the user validates the integer 123 using `is\_valid\_roman` Then the system returns `False` and does not raise an exception.

Implement each criterion as a test. At least one of them must fail on code that already reports 85% branch coverage.

```

tests > test_acceptance.py > test_acceptance_validation_returns_false_for_non_string
1  import pytest
2
3  from roman.converter import (
4      RomanError,
5      from_roman,
6      is_valid_roman,
7  )
8
9
10 def test_acceptance_trims_leading_and_trailing_whitespace():
11     """
12     Given a valid canonical Roman numeral with outer whitespace
13     When the user converts it using from_roman
14     Then the system returns the corresponding integer
15     """
16     assert from_roman(" IV ") == 4
17
18
19 def test_acceptance_rejects_noncanonical_roman_numeral():
20     """
21     Given the noncanonical Roman numeral IIII
22     When the user attempts to convert it using from_roman
23     Then the system raises RomanError
24     """
25     with pytest.raises(RomanError):
26         from_roman("IIII")
27
28
29 def test_acceptance_validation_returns_false_for_non_string():
30     """
31     Given a value that is not a string
32     When the user validates it using is_valid_roman
33     Then the system returns False without raising an exception
34     """
35     assert is_valid_roman(123) is False
  
```